4-Bit CPU

Assembly-Referenz und Funktionsweise

1 Überblick

Diese CPU verarbeitet 4-Bit-Werte (0 bis 15, nur positive ganze Zahlen). Sie besitzt vier Register, einen gemeinsamen Datenbus, eine ALU und ein Flag-Register für Vergleiche. Ein Programm besteht aus höchstens 16 Befehlen. Der Assembler übersetzt Assembly-Text in 14-Bit-Maschinenbefehle.

2 Aufbau der CPU

Register	Adresse	Zweck
`INA`	`00`	Eingangsregister A. Linker Operand jeder ALU-Operation.
`INB`	`01`	Eingangsregister B. Rechter Operand jeder ALU-Operation.
`GPA`	`10`	Allzweckregister, z. B. für Ergebnisse.
`GPB`	`11`	Allzweckregister, z. B. für Ergebnisse.

- **Bus:** Pro Befehl legt genau eine Quelle ihren Wert auf den Bus (ein Register, die Konstante V oder das ALU-Ergebnis). Genau ein Zielregister (RT) übernimmt ihn.
- **ALU:** Rechnet immer mit INA und INB. Ausnahme: NOT verwendet nur INA. Die ALU ändert INA und INB selbst nicht.
- **Flag-Register:** CMP vergleicht INA mit INB und speichert das Ergebnis (kleiner, gleich, größer). Die Sprungbefehle lesen es.
- **Programmzähler:** Läuft Zeile für Zeile. Ein Sprung setzt ihn auf die Adresse V. Die Adresse hat 4 Bit, daher sind die Zeilen 0 bis 15 erreichbar.

3 Maschinenbefehl

Jeder Befehl ist 14 Bit breit und besteht aus vier Feldern:

S5	S4	S3	S2	S1	S0	V3	V2	V1	V0	RS1	RS0	RT1	RT0
OPCODE						**VAR**				**RS**		**RT**	

Feld	Bits	Bedeutung
OPCODE	6	Welche Operation ausgeführt wird. S5 ist 1, wenn der Befehl in ein Register schreibt (SET, MOVE, alle ALU-Befehle), sonst 0.
V	4	Variable: die Zahl bei SET (0-15) oder die Zieladresse bei Sprüngen (0-15).
RS	2	Register Select: Quellregister, das auf den Bus schreibt (bei MOVE).
RT	2	Register Target: Zielregister, das den Wert vom Bus übernimmt.

Nicht benötigte Felder sind immer 0. Beispiel: `SET 13 INB` wird zu `111111 1101 00 01` (OPCODE `111111`, V = 1101, RS = 00 ungenutzt, RT = 01 für INB).

4 Befehlssatz

Befehl	Nr.	OPCODE	Syntax	Wirkung
SET	31	111111	SET <0-15> <Reg>	Ziel = Zahl. Schreibt die Konstante V in das Zielregister.
MOVE	30	111110	MOVE <Quelle> <Ziel>	Ziel = Quelle. Kopiert ein Register in ein anderes.
ADD	29	111101	ADD <GPA|GPB>	Ziel = INA + INB
SUB	28	111100	SUB <GPA|GPB>	Ziel = INA - INB
AND	27	111011	AND <GPA|GPB>	Ziel = INA AND INB
OR	26	111010	OR <GPA|GPB>	Ziel = INA OR INB
XOR	25	111001	XOR <GPA|GPB>	Ziel = INA XOR INB
NOT	24	111000	NOT <GPA|GPB>	Ziel = NOT INA (INB wird ignoriert)
MUL	23	110111	MUL <GPA|GPB>	Ziel = INA * INB
CMP	22	010110	CMP	Vergleicht INA mit INB und setzt die Flags. Schreibt kein Register.
BLT	0	000000	BLT <Zeile|Label>	Springt, wenn INA < INB (laut letztem CMP).
BGT	1	000001	BGT <Zeile|Label>	Springt, wenn INA > INB.
BNE	2	000010	BNE <Zeile|Label>	Springt, wenn INA != INB.
BEQ	3	000011	BEQ <Zeile|Label>	Springt, wenn INA = INB.
JMP	4	000100	JMP <Zeile|Label>	Springt immer (unbedingt).
HALT	5	000101	HALT	Stoppt die CPU.
NOP	6	000110	NOP	Tut nichts (No Operation).

Nr. ist die Nummer aus der Befehlsliste (untere 5 Bit des Opcodes). Der Assembler setzt S5 automatisch.

5 Assembly-Syntax

- Ein Befehl pro Zeile. Groß-/Kleinschreibung ist egal. Operanden werden durch Leerzeichen getrennt.
- Kommentare beginnen mit `;`, # oder `//`. Leerzeilen und Kommentarzeilen zählen nicht als Programmzeile.
- Register heißen `INA INB GPA GPB`. Zahlen sind dezimal (`13`), hexadezimal (`0xD`) oder binär (`0b1101`) erlaubt.
- **Zeilennummern bei Sprüngen zählen ab 1** (1 bis 16). Der Assembler schreibt die CPU-Adresse (0 bis 15) in den Maschinencode. `JMP 1` springt also zur ersten Befehlszeile.
- **Labels** (`name:`) markieren eine Zeile. Sprungbefehle können statt einer Zahl den Labelnamen verwenden. Das ist weniger fehleranfällig.

6 Regeln

- **ALU-Ergebnisse nie nach INA oder INB.** `ADD INA` ist ein Fehler (Feedback-Schleife). `MOVE GPA INA` ist dagegen erlaubt.
- **Vor jedem bedingten Sprung (BLT, BGT, BNE, BEQ) muss ein CMP stehen.** Der Assembler warnt, wenn keines vorher im Quelltext steht.
- **Alle Rechnungen laufen über INA und INB.** Zahlen zuerst mit SET oder MOVE in diese Register bringen, dann die Operation ausführen.
- **SET akzeptiert nur 0 bis 15.** Negative Zahlen gibt es nicht.
- **Maximal 16 Befehle** und Sprungziele 1 bis 16. Mehr lässt die 4-Bit-Adresse nicht zu.
- **Jedes Programm mit HALT beenden.** Ohne HALT läuft der Programmzähler über das Programmende hinaus.

7 Programmiermuster

Addition

Zahlen in INA und INB laden, rechnen, Ergebnis in GPA ablegen. Ergebnis: GPA = 8.

```
SET 5 INA
SET 3 INB
ADD GPA
HALT
```

Register tauschen

GPA und GPB haben keinen direkten Tausch. Die Werte laufen über INA und INB.

```
MOVE GPA INA
MOVE GPB INB
MOVE INA GPB
MOVE INB GPA
HALT
```

Bedingung (if / else)

CMP, dann bedingter Sprung. Trifft die Bedingung nicht zu, läuft das Programm in die nächste Zeile weiter. Ergebnis: GPA = 1.

```
SET 3 INA
SET 7 INB
CMP
BLT kleiner
SET 0 GPA        ; Zweig: INA >= INB
HALT
kleiner:
SET 1 GPA        ; Zweig: INA < INB
HALT
```

Schleife: Summe 5+4+3+2+1

Der Zähler läuft in GPB, die Summe in GPA. Nach 5 Durchläufen ergibt sich GPA = 15 und GPB = 0. Das Beispiel zeigt auch den typischen Ablauf: Werte in INA/INB bringen, rechnen, zurückschreiben.

```
SET 0 GPA        ; Summe
SET 5 GPB        ; Zaehler
loop:
MOVE GPB INA     ; INA = Zaehler
MOVE GPA INB     ; INB = Summe
ADD GPA          ; Summe = Summe + Zaehler
SET 1 INB
SUB GPB          ; Zaehler = Zaehler - 1
MOVE GPB INA
SET 0 INB
CMP              ; Zaehler mit 0 vergleichen
BNE loop         ; weiter, solange Zaehler != 0
HALT
```

Zugehöriger Maschinencode (`--definition`)

```
0000=11111100000010
0001=11111101010011
0010=11111000001100
0011=11111000001001
0100=11110100000010
0101=11111100010001
0110=11110000000011
0111=11111000001100
1000=11111100000001
1001=01011000000000
1010=00001000100000
1011=00010100000000
```

8 Assembler benutzen

```
python assembler.py programm.asm                # Ausgabe auf dem Bildschirm
python assembler.py programm.asm -o out.txt     # in Datei schreiben
python assembler.py programm.asm --definition   # Format AAAA=BBBBBBBBBBBBBB
python assembler.py programm.asm --compact      # ohne Leerzeichen zwischen den Feldern
python assembler.py programm.asm --annotate     # Quellzeile als Kommentar anhängen
```

Bei `--definition` bekommt jede Zeile ihre 4-Bit-Adresse als Prefix, z. B. `0000=11111101010000`. Bei Fehlern (unbekannter Befehl, ungültiges Register, Zahl außerhalb des Bereichs, ALU-Ziel INA/INB) bricht der Assembler mit Zeilennummer ab.

9 Bekannte Einschränkungen

- **Überlauf:** Das Verhalten bei Ergebnissen außerhalb 0 bis 15 (z. B. 15+1, 0-1, 5*5) ist nicht festgelegt. Solche Rechnungen vermeiden, bis es dokumentiert ist.
- **BLT hat den Opcode 000000.** Eine Instruktion nur aus Nullen, etwa leerer Speicher hinter dem Programm, wirkt deshalb wie ein Sprung nach Zeile 1. Programme immer mit HALT beenden.
- **Zeilenlimit:** 16 Befehle. Die Architektur soll später erweitert werden, dann ändern sich Adressbreite und Limits.